```
#include <iostream>
#include <vector>
#include <algorithm>
#include <iomanip>
#include <string>

using namespace std;

struct Process {
    string id;
    int at;     // Arrival Time
    int bt;     // Burst Time
    int ct;     // Completion Time
    int tat;    // Turnaround Time
    int wt;     // Waiting Time
    int index;  // Original index for stable tie-breaking
};

// Comparator to sort primarily by Arrival Time, then by original entry index
bool compareProcesses(const Process& a, const Process& b) {
    if (a.at == b.at) {
        return a.index < b.index;
```

```
    }
    return a.at < b.at;
}

int main() {
    cout << "--- Optimized FCFS CPU Scheduling Simulator ---" << endl;

    int n;
    cout << "Enter the total number of processes: ";
    if (!(cin >> n) || n <= 0) {
        cout << "Invalid number of processes!" << endl;
        return 1;
    }

    vector<Process> processes(n);

    // Accepting user input
    for (int i = 0; i < n; ++i) {
        processes[i].id = "P" + to_string(i + 1);
        processes[i].index = i;

        cout << "\nProcess " << (i + 1) << ":" << endl;
```

```
    cout << "  Enter Arrival Time: ";
    cin >> processes[i].at;
    cout << "  Enter Burst Time: ";
    cin >> processes[i].bt;
}

// Optimization: Sort by Arrival Time while keeping original order for ties
sort(processes.begin(), processes.end(), compareProcesses);

int current_time = 0;
double total_wt = 0;
double total_tat = 0;

// Calculating scheduling metrics
for (int i = 0; i < n; ++i) {
    // Handle CPU idle time if the next process arrives later than current time
    if (current_time < processes[i].at) {
        current_time = processes[i].at;
    }

    processes[i].ct = current_time + processes[i].bt;
    processes[i].tat = processes[i].ct - processes[i].at;
```

```
    processes[i].wt = processes[i].tat - processes[i].bt;

    current_time = processes[i].ct;
    total_wt += processes[i].wt;
    total_tat += processes[i].tat;
}

// Displaying results in a clean formatted table
cout << "\n" << string(65, '=') << endl;
cout << left << setw(10) << "Process"
     << setw(10) << "Arrival"
     << setw(10) << "Burst"
     << setw(12) << "Completion"
     << setw(12) << "Turnaround"
     << setw(10) << "Waiting" << endl;
cout << string(65, '=') << endl;

for (int i = 0; i < n; ++i) {
    cout << left << setw(10) << processes[i].id
         << setw(10) << processes[i].at
         << setw(10) << processes[i].bt
         << setw(12) << processes[i].ct
```

```
            << setw(12) << processes[i].tat
            << setw(10) << processes[i].wt << endl;
    }

    cout << string(65, '=') << endl;

    // Formatting float output to 2 decimal places
    cout << fixed << setprecision(2);
    cout << "\nAverage Waiting Time: " << (total_wt / n) << endl;
    cout << "Average Turnaround Time: " << (total_tat / n) << endl;

    return 0;
}
```